Heap and Heapsort

Performance of certain operations with different data structure

Array Input	Insert	Search	Find Minimum	Delete (Min)
Unsorted array				
Sorted Array				
Unsorted Linked List				
Heap(Min)				

Performance of certain operations with different data structure

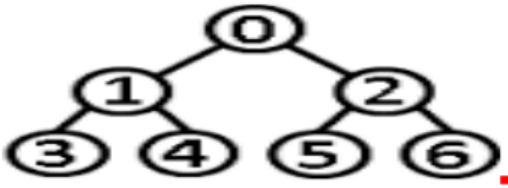
Array Input	Insert	Search	Find Minimum	Delete (Min)
Unsorted array	0(1)	0(n)	0(n)	0(n)
Sorted Array	0(n)	0(log n)	0(1)	0(n)
Unsorted Linked List	0(1)	0(n)	0(n)	0(n)
Heap(Min)	0(log n)	0(log n)	0(1)	0(1)



full
binary tree



complete
binary tree



perfect
binary tree

Binary trees can be categorized based on specific structural properties:

Full Binary Tree:

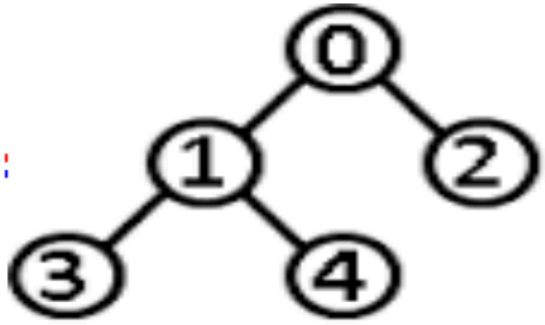
- A binary tree is considered full if every ***node has either zero or two children***.
- In other words, no node in a full binary tree can have only one child.
- All internal nodes (non-leaf nodes) must have exactly two children.

Complete Binary Tree:

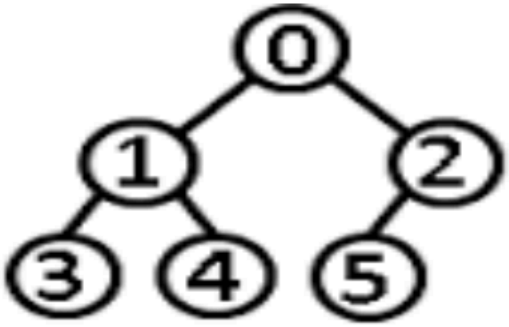
- A binary tree is considered complete if all levels are completely filled, except possibly the last level.
- If the last level is not completely filled, ***all nodes in that level must be as far left as possible***.
- There cannot be any gaps in the filling of nodes from left to right on the last level.

Perfect Binary Tree:

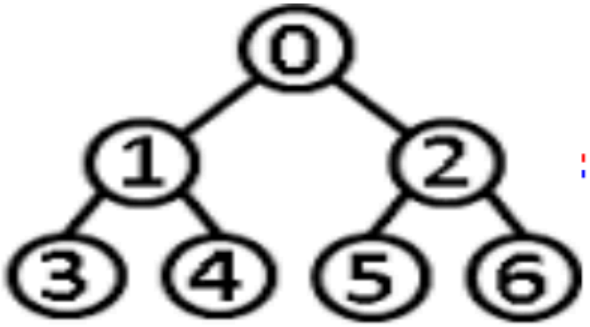
- A perfect binary tree is a special type of binary ***tree that is both full and complete***.
- All internal nodes have exactly two children, and all leaf nodes are at the same level (depth).
- This implies that all levels of the tree are completely filled.



full
binary tree



complete
binary tree



perfect
binary tree



A



B



C



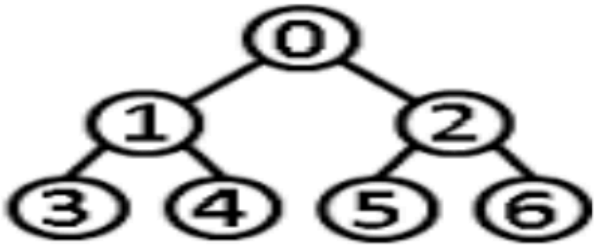
D



full
binary tree



complete
binary tree



perfect
binary tree

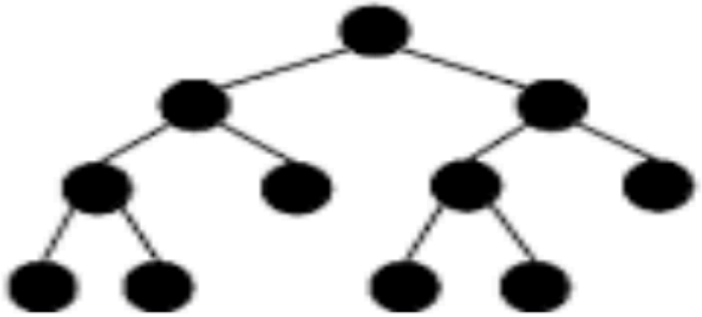
Neither complete nor full



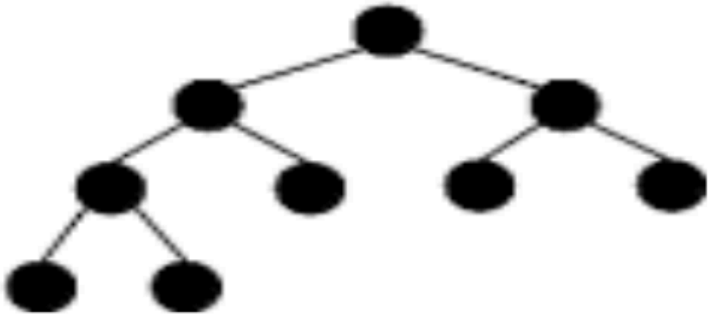
Complete but not full



Full but not complete



Complete and full



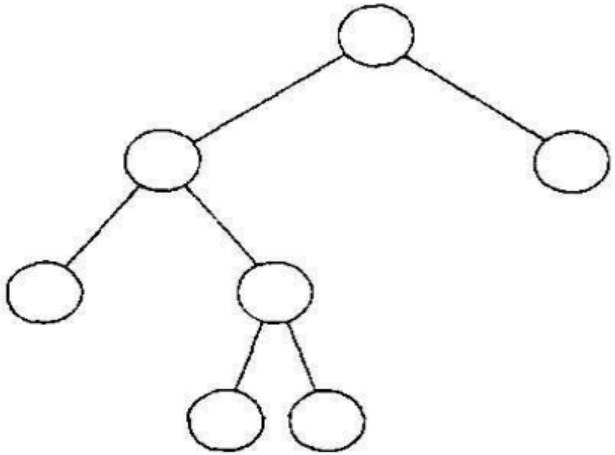
Heap

- A Heap data structure is a binary tree with special properties:
  - **Heap Structure**
  - **Partial order tree property**

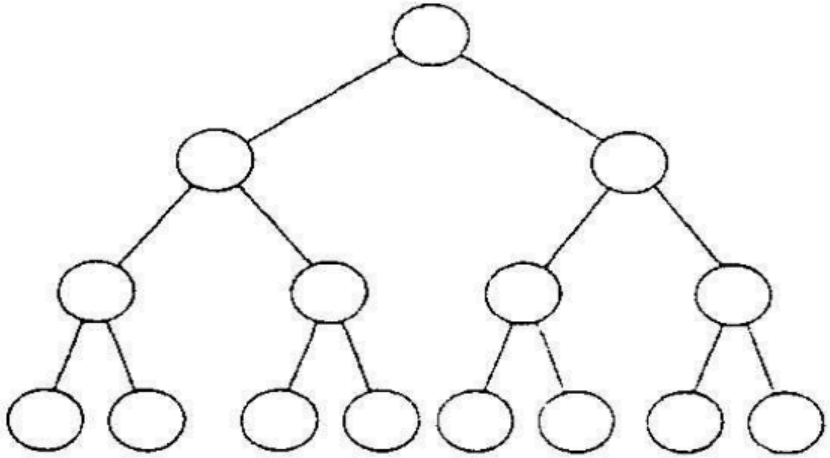
- Definition: Heap Structure
  - **A binary tree T is a heap structure if and only if it satisfies the following conditions: (h = height of the tree)**
    - 1. T is complete
    - 2. All leaves are at depth h or h –1

- Definition: Partial order tree property
  - **A tree T is a (maximizing) partial order tree if and only if the key at any node is greater than or equal to the keys at each of its children (if it has any)**

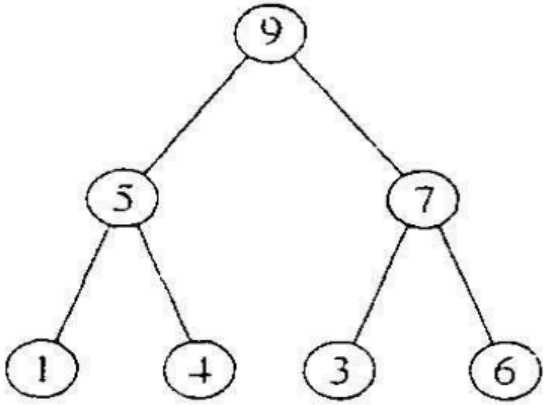
e.g. Heaps (or not)



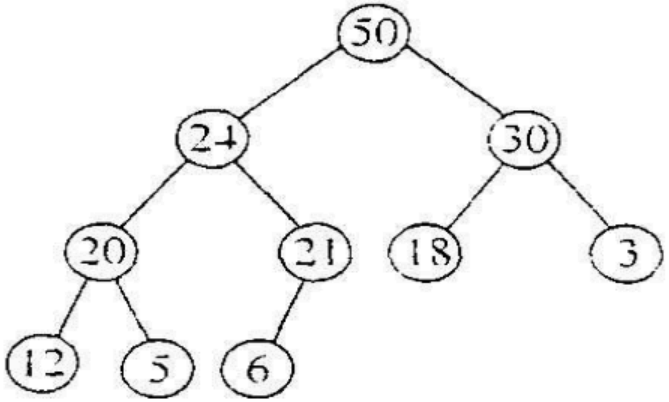
(a) A 2-tree



(b) A complete binary tree



(c) Heap 1



(d) Heap 2

Some Properties

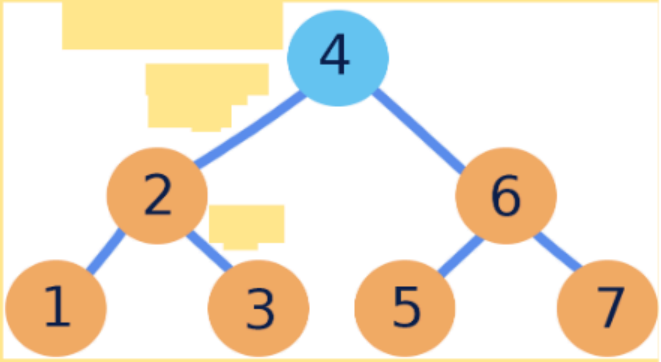
- Height of binary tree (log n)
- Max. number of nodes in complete binary tree of height h is $(2^{h+1} - 1)$
- All leaves in tree are consecutive and starts at [floor(n/2)+1 to n] (for index starts at 1 else [floor(n/2) to n])
- Every node is heap in itself
- Binary heap can be implemented in array using relationship
  - ➢Left_child= 2*index
  - ➢Right_child = 2 *index +1
  - ➢Parent of node = floor (index /2)

If index starts with 0 then

- ➢Left_child= 2*index +1
- ➢Right_child = 2 *index + 2
- ➢Parent of node = ceil(index /2) -1

```
BUILD-MAX-HEAP(A)
{
    A.heap-size= A.length
    for(i=floor(A.length/2) downto 1*)
    MAX-HEAPIFY(A,i)
}
```

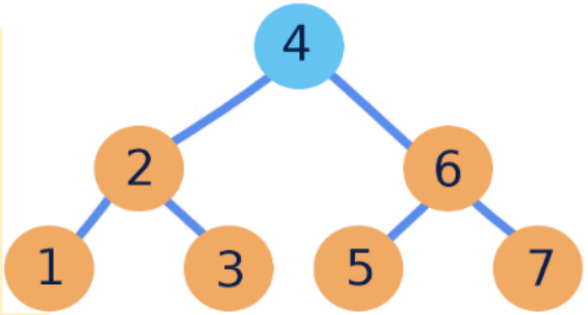
```
MAX-HEAPIFY(A,i)
{
    l=2i
    r=2i+1
     If(l<=A.heap-size and A[l]>A[i])
        largest=l
     else largest = i
     If(r<=A.heap-size and A[r]>A[largest])
        largest=r
     If(largest<>i)
         Exchange A[i] with A[largest]
         MAX-HEAPIFY(A,largest)
}
```



An array with descending Order (complete tree)is Max-Heap
An array with ascending Order (complete tree)is Min-Heap
=> Sorting cost is **n log n**

i=floor(A.length/2) downto 1*) OR
i=floor(A.length/2)-1 downto 0*)

```
MAX-HEAPIFY(A,i)
{
    l=2i
    r=2i+1
     If(l<=A.heap-size and A[l]>A[i])
        largest=l
     else largest = i
     If(r<=A.heap-size and A[r]>A[largest])
         largest=r
     If(largest<>i)
          Exchange A[i] with A[largest]
     MAX-HEAPIFY(A,largest)
}
```



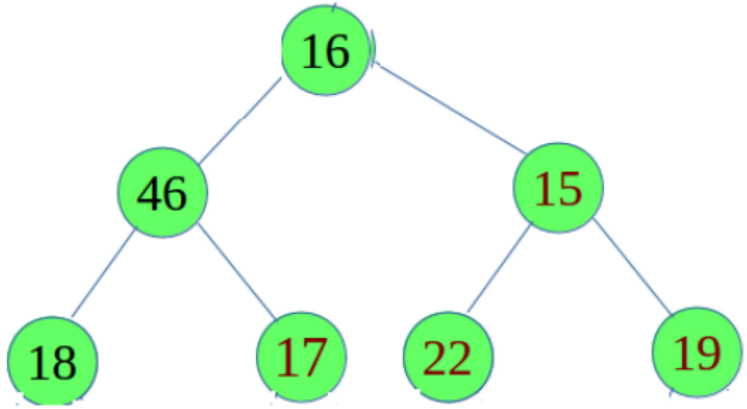
0	1	2	3	4	5	6
4	2	6	1	3	5	7

Heapify (A,0) = ?

0	1	2	3	4	5	6
6	2	7	1	3	5	4

STEPS						
0	1	2	3	4	5	6
4	2	6	1	3	5	7
6		4				
		7				4
6	2	7	1	3	5	4

0	1	2	3	4	5	6
16	**46**	**15**	**18**	**17**	**22**	**19**



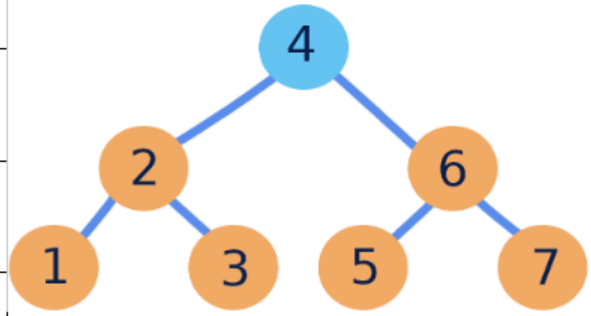
STEPS						
0	1	2	3	4	5	6
16	**46**	**15**	**18**	**17**	**22**	**19**

Heapify (A,0) = ?

```
BUILD-MAX-HEAP(A)
{
    A.heap-size= A.length
    for(i=floor(A.length/2) downto 1*)
    MAX-HEAPIFY(A,i)
}
```

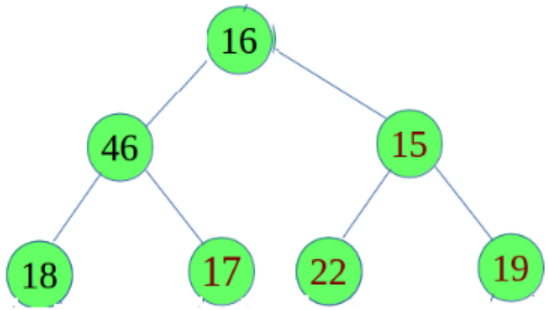
i=floor(A.length/2) downto 1*) OR
i=floor(A.length/2)-1 downto 0*)

i → 2 to 0		**0**	**1**	**2**	**3**	**4**	**5**	**6**	
		4	**2**	**6**	**1**	**3**	**5**	**7**	
i=2	i=2 Heapify(A,2)		**Swap A[2] <->A[6]**						**i=2** **l=5, r=6**
	Heapify(A,6)			**7**				**6**	Exit-heapify(A,6)
		4	**2**	**7**	**1**	**3**	**5**	**6**	
i=1	i=1 Heapify(A,1)	**4**	**2**	**7**	**1**	**3**	**5**	**6**	**i=1** **l=3, r=4**
			Swap A[1] <->A[4]						
	Heapify(A,4)		**3**			**2**			**i=4** **l=9, r=10 (X)**
		4	**3**	**7**	**1**	**2**	**5**	**6**	Exit-heapify(A,4)
i=0	i=0 Heapify(A,0)	**4**	**3**	**7**	**1**	**2**	**5**	**6**	**i=0** **l=1, r=2**
			Swap A[0] <->A[2]						
	Heapify(A,2)	**7**		**4**					**i=2** **l=5, r=6**
		7	**3**	**4**	**1**	**2**	**5**	**6**	
			Swap A[2] <->A[6]						
				6				**4**	Exit-heapify(A,4)
	Heapify(A,6)	**7**	**3**	**6**	**1**	**2**	**5**	**4**	



i → 2 to 0	0	1	2	3	4	5	6	
	16	46	15	18	17	22	19	

i=floor(A.length/2) downto 1*) OR
i=floor(A.length/2)-1 downto 0*)



0	1	2	3	4	5	6
7	3	6	1	2	5	4

```
Heap-Extract-Max(A)
{
        If A.heap-size < 1
            error " heap underflow"
        max=A[1]
        A[1]=A[A.heap-size]
        A.heap-size=A.heap-size-1
        MAX-HEAPIFY(A,1)
        return max
}
```

STEPS						
0	1	2	3	4	5	6
7	3	6	1	2	5	4
						4
4 i	3 l	6 r	1	2	5	
6		4				
6	3	4 i	1	2	5 l	r
		5			4	
6	3	5	1	2	4	

Complexity Analysis

Max-Heapify

Heap-Extract-Max

Build_Heap

Performance Analysis

Max-Heapify takes O(log n)
Heap-Extract-Max running time is O(log n)
The running time of Build_Heap is O(n)

```
Heap-Extract-Max(A)
{
        If A.heap-size < 1
            error " heap underflow"
        max=A[1]
        A[1]=A[A.heap-size]
        A.heap-size=A.heap-size-1
        MAX-HEAPIFY(A,1)
        return max
}
```

- Number of at most node with height h ceil($n/2^{h+1}$)
- Time taken by any Max-Heapify at any node is O(h)

```
BUILD-MAX-HEAP(A)
{
        A.heap-size= A.length
        for(i=floor(A.length/2) downto 1)
        MAX-HEAPIFY(A,i)
}
```

```
MAX-HEAPIFY(A,i)
{
    l=2i
    r=2i+1
     If(l<=A.heap-size and A[l]>A[i])
        largest=l
     else largest = i
     If(r<=A.heap-size and A[r]>A[largest])
        largest=r
     If(largest<>i)
        Exchange A[i] with A[largest]
        MAX-HEAPIFY(A,largest)
}
```

Heap sort

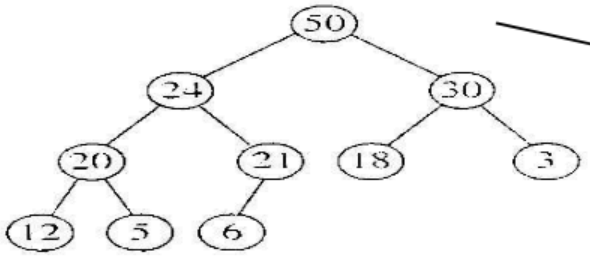
Heapsort Strategy

- The elements to be sorted are arranged in a heap
- With this heap, we can build a sorted sequence in reverse order by repeatedly removing the element from the root,
- Rearranging the remaining elements to reestablish the partial order tree property, and so on.

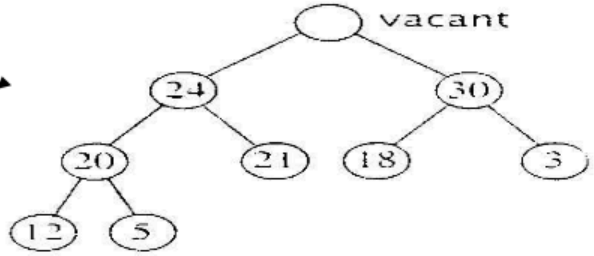
How does it work?(Max-Heapify/Build-Max-Heap/Heap-Extract-Max)

Note
Ascending and descending will always be min and max heap respectively

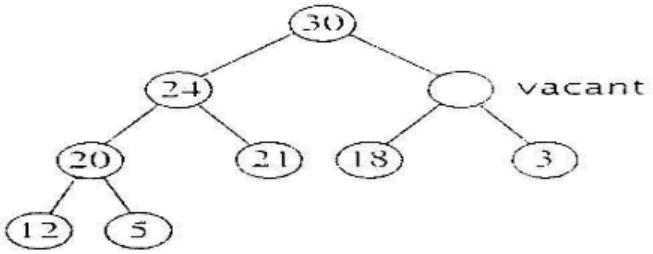
Heapsort in action



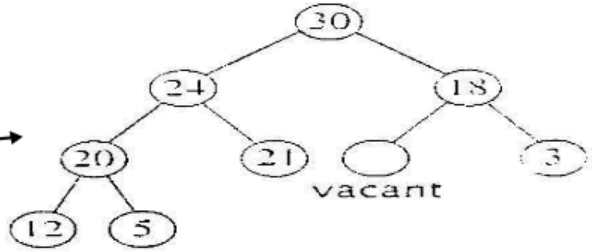
(a) The heap



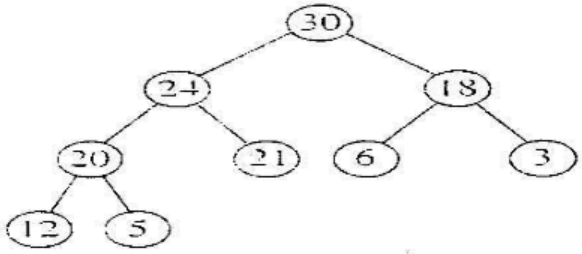
(b) The key at the root has been removed; the rightmost leaf at the bottom level has been removed. $K = 6$ must be reinserted.



(c) The larger child of vacant, 30, is greater than K, so it moves up and vacant moves down.



(d) The larger child of vacant, 18, is greater than K, so it moves up and vacant moves down.



(e) Finally, since vacant is a leaf, $K = 6$ is inserted.

HEAP-SORT(A)

BUILD-MAX-HEAP(A)

For I ← length [A] down to 2

Do exchange A[1] □□ A [i]

Heap-size[A] □ heap-size[A]-1

MAX-HEAPIFY(A,1)

The running time of Build_Heap is O(n)
Max-Heapify takes O(log n)
(heapify to apply n-1 times)

Therefore running time is **O(n log n)**

```
BUILD-MAX-HEAP(A)
{
        A.heap-size= A.length
        for(i=floor(A.length/2) downto 1)
        MAX-HEAPIFY(A,i)
}
```

```
HEAP-SORT(A)
    BUILD-MAX-HEAP(A)
    For I ← length [A] down to 2
        Do exchange A[1] □□ A [i]
            Heap-size[A] □ heap-size[A]-1
            MAX-HEAPIFY(A,1)
```

Build Max heap	0	1	2	3	4	5
i→ 2 to 0	**12**	**7**	**8**	**23**	**3**	**21**
for i→2			i			L
exit			21			8
	12	7	21	23	3	8
for i→1		i		L	R	
exit		23		7		
	12	23	21	7	3	8
for i→0	i	L	R			
	23	12				
	for i→1	i		L	R	
exit	23	12	21	7	3	8

Heap-Sort	0	1	2	3	4	5
It-1	**23**	**12**	**21**	**7**	**3**	**8**
	8					23
heapify	i	L	R			
	21	12	8	7	3	23
exit			i			
	21	12	8	7	3	23
It-2	3				21	23
heapify	i	L	R			
	12	3	8	7	21	23
heapify		i		L		
	12	7	8	3	21	23
exit				i		
It-3	12	7	8	3	21	23
	3			12	21	23
heapify	i	L	R			
	8		3			
exit			i			
It-4	8	7	3	12	21	23
	3	7	8	12	21	23
heapify	i	L				
exit	7	3				
It-5	7	3	8	12	21	23
	3	7	8	12	21	23



Video Lecture of the topic can be found at

Heap – Introduction

https://www.youtube.com/watch?v=phYfC_NqfHQ

Illustrative Example of MaxHeapify BuildMax ExtractMax Procedures

https://www.youtube.com/watch?v=iCQO2OP1_TI&t=5s

Build Max Heapify ExtractMAx Analysis

https://www.youtube.com/watch?v=qcoqre-JHQU

HeapSort Strategy and Complexity Analysis

https://youtu.be/jCKWkFTZwCk